

**Oxigraph**

# Oxigraph TL;DR

Created in 2017, first release in 2020

Goal: not too slow RDF/SPARQL stack without the JVM

Design:

- modular Rust crates (parsers, SPARQL evaluation...)
- Python and JS/WASM bindings (performances ~10x rdflib)
- Disk storage with RocksDB (a key-value store)

# Current state: the plus

Increasingly robust utilities:

- Turtle, TriG, RDF/XML, NTriples, NQuads, JSON-LD (streaming!), SPARQL parsers and serializers
- SPARQL evaluator (volcano-style)

For next release:

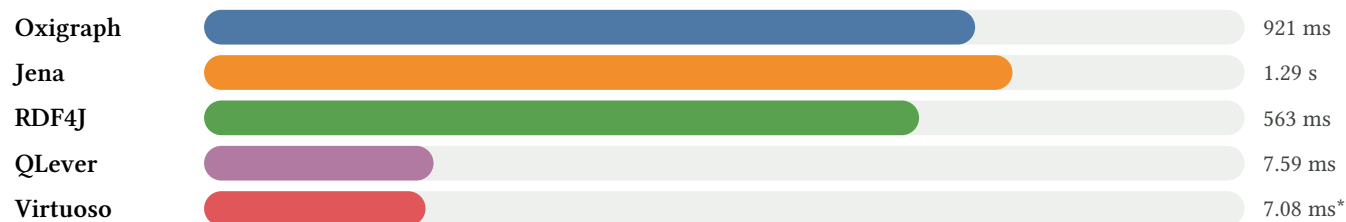
- faster parsers
- more correct SPARQL

# Development practices

- W3C testsuite format is great for testing (factorize boilerplate!)
- Fuzzers are great to check invariants:
  - Parsers do not crash
  - Serialization+parsing is idempotent
  - The query optimizer don't change the results
  - ...
- Micro-benchmarks in the CI to catch regressions
- Getting used to LLMs

# Benchmark

## Q1–Q5 · read workload (geometric mean; Virtuoso: Q5 missing)



## Q6 · DELETE–INSERT prices (QLever and Virtuoso: unavailable)



*DataTreeHouse Trainmarks benchmark XLarge - 10M as of 2026-09-13*

# Current state: the minus

Naive and slow SPARQL query execution

- basic query optimizer
- one row at a time
- naive joins

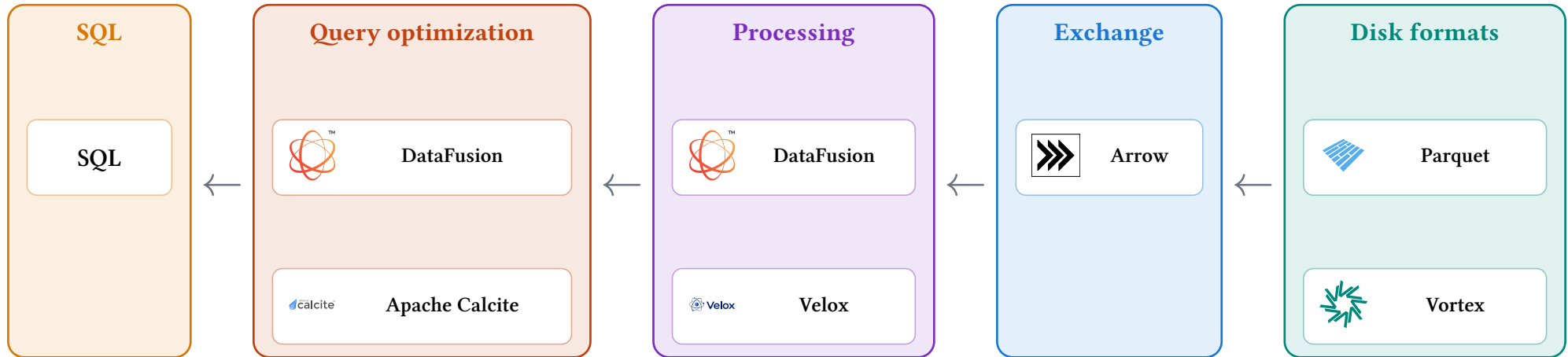
The RocksDB storage is great for writes but not for analytical reads

One guy project without enough time...

What can we leverage to fix that?

# Composable OLAP stack

Composable columnar query execution stack for analytics:



# Is it relevant? SPARQL vs SQL

With a quad table:

- triple pattern become a table read with filters
- BGPs become joins
- Other operators map fairly well (OPTIONAL, UNION, GROUP BY)

# SPARQL vs SQL issues

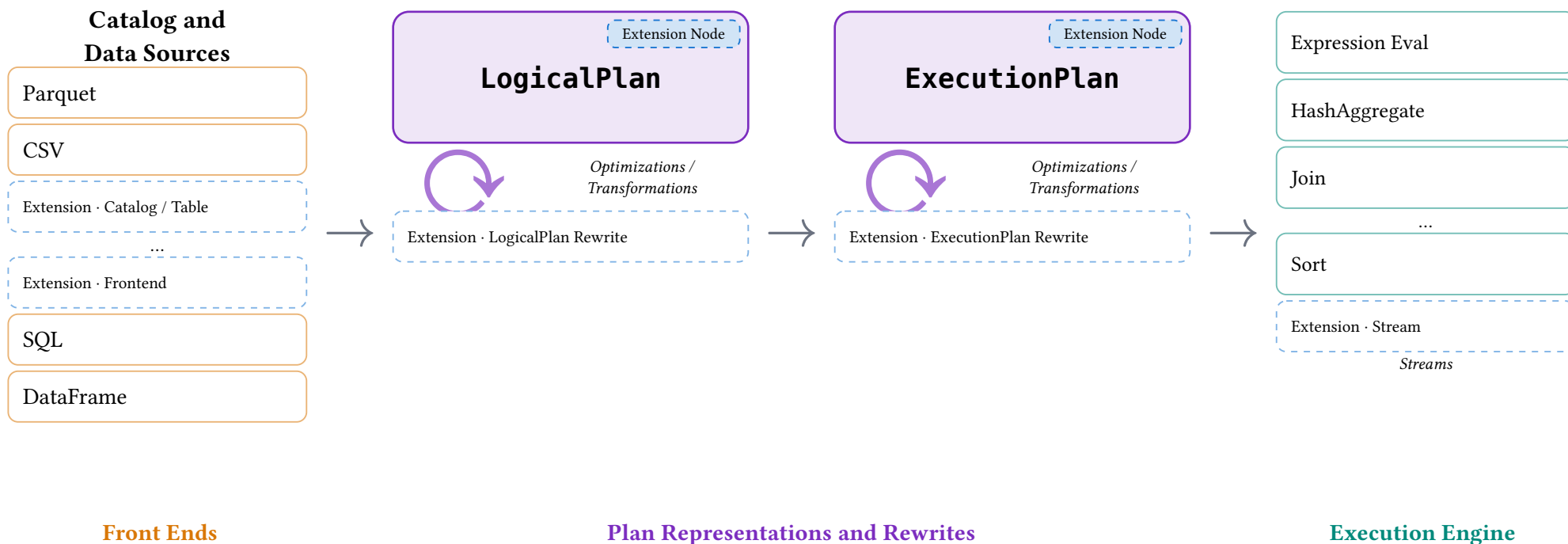
“Fun” parts:

- SPARQL terms are untyped
- Functions and operators
- Hit “niche” features like lots of self-joins and `WITH RECURSIVE` for property paths

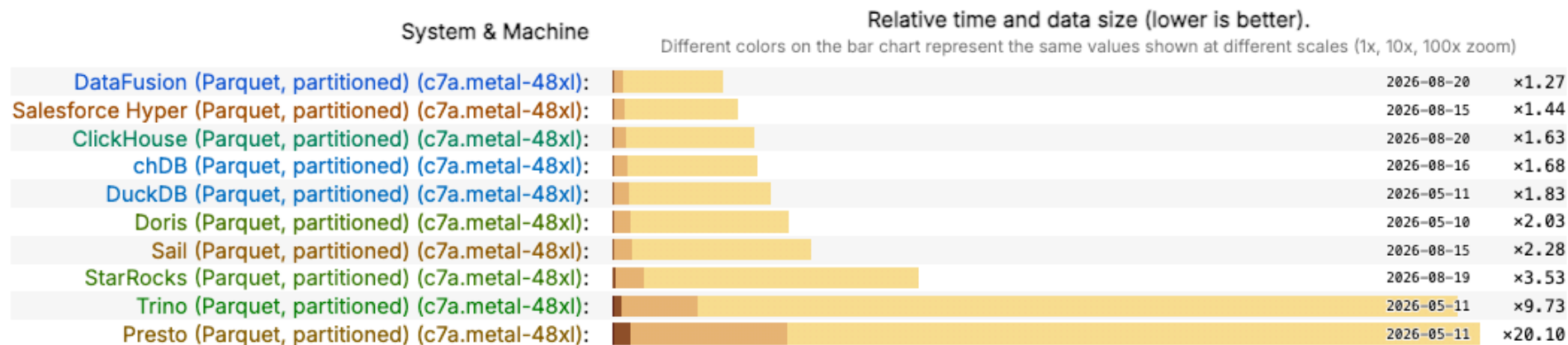
but basic operators stay the same!

# Apache DataFusion

Modular SQL implementation on top of Arrow written in Rust



# Performances?



ClickBench results for c7a.metal-48xlarge as of 2026-08-24

# Can we leverage DataFusion for SPARQL?

- Custom “data sources” for RDF-optimised storages
- Convert SPARQL to a standard query plan with custom functions
- Tailored optimization passes

We get for (nearly) free:

- Lots of generic query optimization/evaluation work
- R2RML with connectors to Iceberg, PostgreSQL...

First implementation by T. Schwarzingner, “RDF fusion”. Already a branch in Oxigraph.

# Open challenge: fitting RDF in Arrow

Arrow supports SQL-like types (string, float, structs, lists)

For terms:

- Serialized as strings (bad for speed)
- Dictionary encoding (needs a global dictionary + “fun” to orchestrate dictionary lookups)
- Struct-based encoding (kind, literal datatype/language tag/..., quite complex)

Would love to read a paper comparing approaches and their thread-offs!

# Open challenge: storage format

If we can emit Arrow we can use whatever we want

- Still rotated (SPO, POS, OSP) trees?
- Something WCO joins friendly?
- What about updates?

# What about the existing systems (Ontop, TinySPARQL...)?

Design: They rewrite SPARQL to SQL for some existing database.

Consequence: They can't customize deeply the DB (bespoke functions, optimizer, storage...)

But: With DataFusion we can customize all of these!

# Conclusion

Two topics I would love to read more on:

- *practical* large scale SPARQL execution
- SPARQL-RL practicalities and interactions with query/update workloads

Also, help welcome!

## References:

- *RDF Fusion: An Extensible SPARQL Engine for Hybrid Data Models*, Schwarzhinger, T., Thoma, M., Preindl, T., Kjær, M., Just, V. P., & Steindl, G.
- *Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine*, Lamb, A., Shen, Y., Heres, D., Chakraborty, J., Kabak, M. O., Hsieh, L. C., & Sun, C.
- *The Composable Data Management System Manifesto*, Pedreira, P., Erling, O., Karanasos, K., Schneider, S., McKinney, W., Valluri, S. R., ... & Nadeau, J.
- *COTTAS: Columnar Triple Table Storage for Efficient and Compressed RDF Management*, Arenas-Guerrero, J., & Ferrada, S.